PROJECT WRITE-UP · GROUP 14

Calgary Residential Development ROI Classifier

CMPT 310 – D100 Introduction to Artificial Intelligence, Summer 2026

Finley Thomson · Monem Moheq · Srinivas Suggu · Jarrod Hwang

1. System Explanation

Our group built an offline supervised-learning pipeline that classifies completed Calgary residential developments as Low, Medium, or High ROI. Its intended use is research and early-stage triage: ranking projects for deeper expert review. It is not a deployed valuation service and must not be used as financial advice.

Input, Target, and Output

The final modeling table combines building-permit and historical assessment records to determine a proxy for ROI and with community-income and proximity-based features. Application years span 2017–2023 and completion years span 2018–2023. The classes are nearly balanced: 73 Low, 72 Medium, and 73 High.

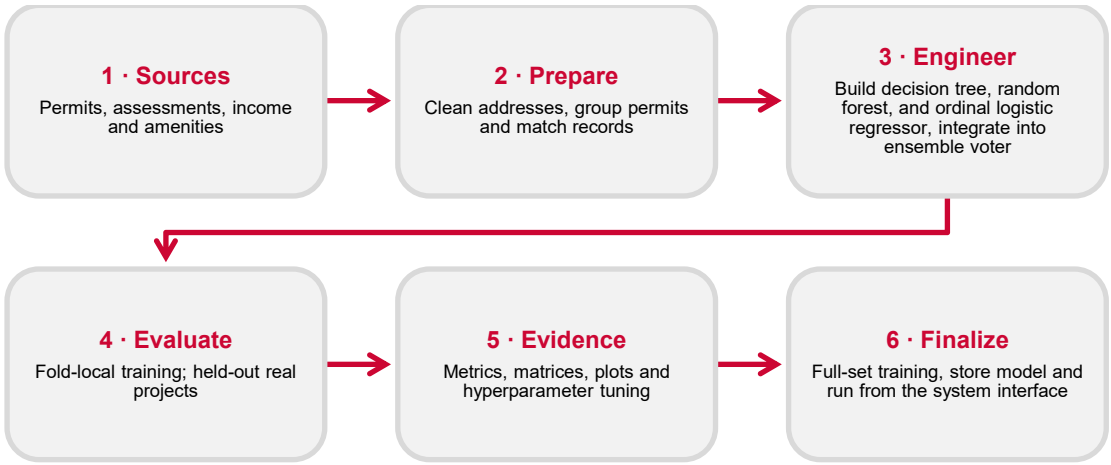
$$\text{ROI} \approx \frac{(FinalAssessment - (InitialAssessment + EstimatedDevelopmentCost))}{(InitialAssessment + EstimatedDevelopmentCost)}$$

The system interface takes 4 inputs namely, an address in Calgary, a Calgary address, a purchase price, and an estimated number of units to be built. From this, the system then calculates seven values available before the outcome: initial assessment, community income, cost per unit, distance to downtown, and distance to the nearest transit stop, park, and public school, passing these to the classifier. Final assessment, ROI, and Classification are explicitly excluded from prediction features.

AI Methods

Our system leverages our random forest classifier and ordinal logistic regressor to form an ensemble voter, utilizing a soft voting system to determine classifications on data points provided by the user through the system interface. In addition to the main system interface, available is the ability to compare our custom-made models' evaluations against each other and optionally with those of a majority-class baseline and a scikit-learn random forest via 5-fold stratified cross-validation, generating new synthetic data from each fold's training set to ensure no data leakage.

AI Pipeline



Evaluation Results

Running the aforementioned evaluations of our models, during the cross-validation, each real project appears once in held-out evidence, producing 224 out-of-fold predictions. The performance of these models is then captured and transformed into easily digestible formats as seen below. The results of this final evaluation on the ensemble and baseline are in the following table.

Model	Training	Training Accuracy	Accuracy	Training F1	Macro-F1	Ordinal MAE	Severe
Majority baseline	Real only	33.7%	32.6% ± 0.9%	16.8%	16.4 ± 0.3%	1.018	75
Ensemble	Real only	61.1%	45.4% ± 4.0%	60.9%	45.0% ± 4.0%	0.752	45
Ensemble	+1,200/fold for RF Real only for OLR	49.1%	40.8% ± 5.0%	48.3%	39.3% ± 4.9%	0.794	44

Model	Training	Training Accuracy	Accuracy	Training F1	Macro-F1	Ordinal MAE	Severe
Ensemble	Real only for RF +1,200/fold for OLR	49.8%	45.8% ± 8.3%	48.9%	45.5% ± 8.8%	0.761	48
Ensemble	+1,200/fold for RF +1,200/fold for OLR	52.7%	41.7% ± 6.9%	52.3%	41.3% ± 7.1%	0.803	48

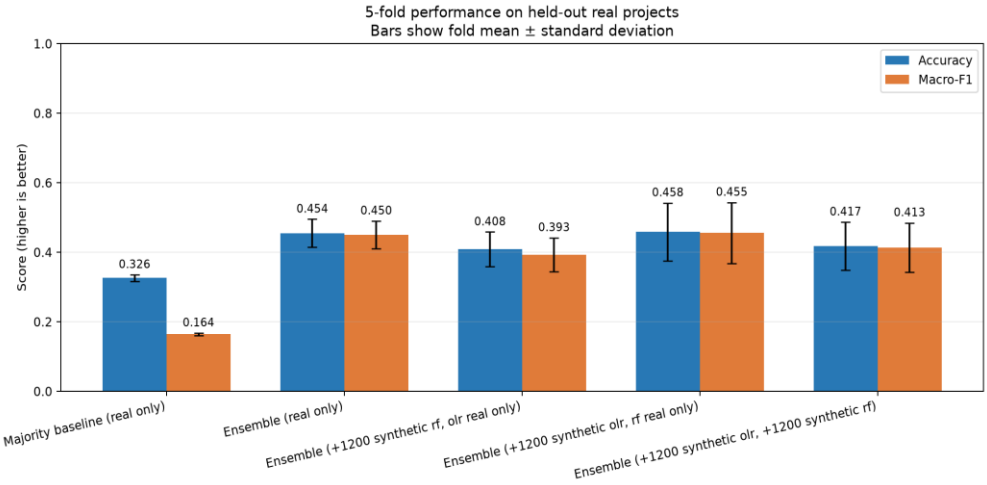


Figure 1. Five-fold mean ± standard deviation on held-out real projects.

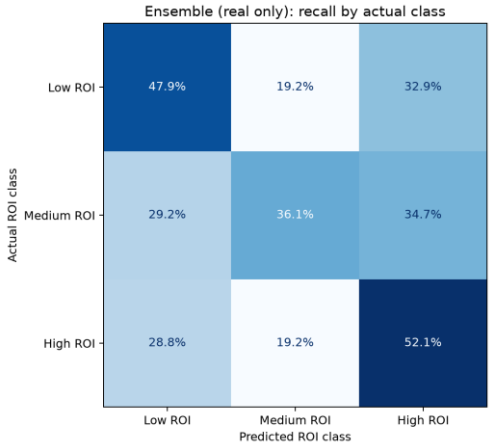


Figure 2. Row-normalized confusion matrix for the real-only ensemble voter; diagonal cells are class recall

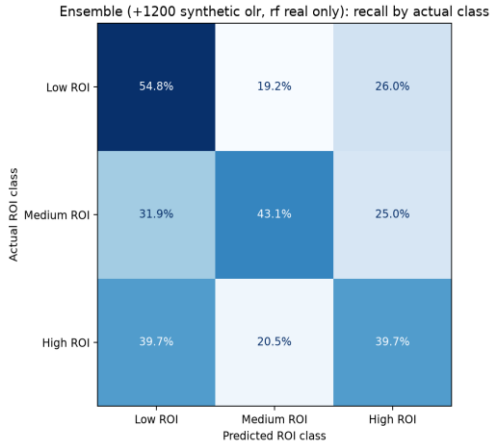


Figure 3. Row-normalized confusion matrix for the 1,200 synthetic OLR, real-only RF ensemble voter; diagonal cells are class recall

Interpretation

The real-only ensemble and +1,200 synthetic OLR and real-only ensemble are the two best models by a notable margin. They improve accuracy by 12.8% and 13.2% respectively, and macro-F1 by 29.0% and 29.4% respectively over the baseline. Recall remains modest being 47.9% and 54.8% (respectively) for “Low”, 36.1% and 43.1% (respectively) for “Medium”, and 52.1% and 39.7% (respectively) for “High”.

The notable difference between the two models is that the spread of the accuracy and F1 for the real-only model is less than half of the corresponding spreads for the 1,200 synthetic OLR, real-only RF model, meaning that the real-only, although yielding marginally worse metric scores, could be more reliable, averaging the model results of more seeded cross validation runs would be necessary to confirm this. Additionally, the real-only model balances out the predictions between “High” and “Low” more evenly but missing much of the “Medium” predictions; this contrasts with the 1,200 synthetic OLR, real-only RF model which leans more towards “Low” predictions at the expense of missing many “High” predictions. In turn, this causes less of the most critical errors (a prediction of “High” for projects that are actually “Low”) but increasing the number of missed opportunities (projects actually “High” but predicted as “Low”). This can be interpreted as the real-only model being a somewhat more “aggressive” predictor, favouring “High” predictions (although it still retains good recall for “Low” predictions) and the 1,200 synthetic OLR being a “lower-risk” model, erring on the side of caution and favouring “Low” predictions, thus avoiding bad investments.

Ultimately, we see that the synthetic-data hypothesis succeeded in a certain sense, and failed in another. Adding 1,200 generated rows per fold into the training of the ordinal logistic regression resulted in the highest average score, with the best fold having a score

of 50%, in which sense the synthetic-data hypothesis succeeded. However, the model retained the greatest spread across both accuracy and F1, as well as predicting far more “Low” projects than “High” as compared to the real-only model, in which sense the hypothesis failed.

Limitations

Unfortunately, due to a lack of further, complete data available, alongside a highly limited universe of real-world data (given there are only so many townhomes in Calgary), the model is undercut by lacking a chance to discern broader patterns. Additionally, the features we have elected to use to glean an ROI classification, may simply not yield a strong enough connection, despite their intuition. Furthermore, real estate development is inherently risky and therefore noisy and unpredictable, as there are many factors that influence a project’s ROI which cannot be predicted prior to development (e.g. market swings, extreme weather events, immigration legislature, etc.). Ultimately, the models’ ability to more accurately predict the ROI can be attributed to the small, noisy, unpredictable real estate development dataset.

Risk Mitigation

In an attempt to mitigate these problems we created synthetic data for training, as mentioned before. , This was done through the generation of synthetic data following a multivariate, lognormal distribution, although this lacked much merit in the end. Even still one potential issue with this approach is the model fitting to the lognormal distribution of the synthetic data, rather than the underlying patterns within the real data. Thus we must be careful with the use of this data, particularly for the random forest, as decision trees tend to fit too closely to training data.

2. Feature table

Completeness follows the rubric’s 0–5 scale. Author cells list contributors responsible for substantial repository implementation of the submitted component. It should be noted that all code used was 100% original code, names are ordered by amount contributed.

Description	Platform	Complete	Code	Author(s)	Notes
Data gathering, organizing and, filtering	Local	5/5	Python	Finley Thomson, Monem Moheq	Building permits, assessment reports, and other municipal datasets are gathered and sorted through.
Data reconciliation and synthesization	Local	5/5	Python	Finley Thomson, Monem Moheq	Combine all relevant data across all datasets into final 224 townhouse project dataset.
Decision tree and random forest classifiers	Local	4/5	Python	Finley Thomson, Jarrod Hwang	Built decision tree and random forest ensemble from scratch, working and reproducible but has limited boundary coverage.
Ordinal Logistic Regressor	Local	4/5	Python	Finley Thomson, Monem Moheq	Built ordinal logistic regressor from scratch, working and reproducible but needs work to ensure proper threshold ordering.
Ensemble Voter Infrastructure	Local	5/5	Python	Finley Thomson	Built ensemble infrastructure to engage other models in a soft voting structure.
Synthetic Data Generation	Local	4/5	Python	Finley Thomson, Jarrod Hwang	Integrated synthetic data generation based off a multivariate lognormal distribution; more confirmation needed for validity of the distribution.
Leakage-Safe Stratified Cross-Validation	Local	5/5	Python	Finley Thomson, Jarrod Hwang	Five folds, real held-out evidence, fold-local generation, and deterministic configuration.
Compare and Evaluate Models (Including sklearn Random Forest and Baseline)	Local	5/5	Python	Jarrod Hwang, Finley Thomson	Canonical CLI reports accuracy, macro-F1, ordinal error, and severe errors.
Export evaluation evidence	Local	5/5	Python	Jarrod Hwang	Produces JSON/CSV summaries, class/fold metrics, matrices, plots, hash, and manifest.
Hyperparameter Tuning and Final Training	Local	5/5	Python	Finley Thomson	Performing a random grid search to attempt to find optimal hyperparameter values before training a storing models for interfacing.
Website Interface Design	Local	5/5	HTML, CSS, and JavaScript	Finley Thomson	Creating and designing website interface for ease of user input.
Website Integration	Local	5/5	Python, JavaScript, HTML	Finley Thomson	Integration of website interface with models, converts address, community, purchase price, and est. units to full data point to predict and display ROI.

Completeness scale applied

5 · Complete Working, tested, integrated, and clear for the submitted scope

4 · Working Integrated with minor coverage, polish, or reproducibility limits.

3 · Partial: demonstrates the concept but needs manual or non-portable steps.

>2 · Not Finalized prototype, research, or unimplemented; not claimed as final.

Submitted code organization

```
CMPT310_Group14_Code_Submission/
├── Data/Code/              # fold-local synthetic generator
├── Data/CSVs/Sorted/       # final modeling table
├── Model/                  # evaluator, CV, metrics, custom tree/forest
├── website/
├── ExampleSites.txt        # for demonstrating and using the website
├── requirements*.txt
├── RunROIClassifier.py
└── README.md
```

Practical software-quality assessment

The user-facing web interface boasts many practical features for usage, including:

Functional Interface The visual web interface works properly, allowing users to input requisite information and predict using the stored models.	**Reliability** Pre-trained and stored models ensure consistency of predictions across inputs.	**Performance** Stored, pretrained models allow for a novel point to be predicted quickly.
Maintainability Evaluation responsibilities are separated; raw preprocessing remains outside the clean run path.	**Compatibility** Runs locally on Python 3.11+ from the included final CSV; exact tested versions are supplied.	**Security and usability** No credentials or student IDs are included; explicit warnings reduce financial risk.

As for the user-facing web interface boasts many practical features for usage, including:

Functional Interface Functional suitability Canonical evaluation, baseline, forests, metrics and artifacts work; no live inference UI is claimed.	**Reliability** . Fixed seeds and held-out evidence support repeatability; the small dataset creates high variance.	**Performance** Parallel processing of trees and vectorized gradient computations provide fair performance in training models while still using python.
Maintainability Evaluation responsibilities are separated and documented; raw preprocessing remains outside the clean run path.	**Compatibility** Runs locally on Python 3.11+ from the included final CSV; exact tested versions are supplied.	**Security and usability** No credentials or student IDs are included; explicit warnings reduce financial risk.

3. External Tools, Libraries, and Data

Software used by the submitted run path

Description	Platform
NumPy	Arrays, vectorized math operations, normal + covariance matrices.
Pandas	CSV I/O, validation, tabular features.
Scikit-learn	Indices for train/test splits, standard scaling, baseline comparison model, metrics.
Matplotlib	Comparison and confusion-matrix figures.
Flask	Framework to implement web interface
Haversine	Calculating distances between coordinates.

Joblib	Storing trained models int pkl files.

External packages are used through their documented public APIs. The submitted final path does not copy an external model implementation; the custom tree/forest source is included for inspection.

Data sources

- City of Calgary, **Building Permits**, dataset c2es-76ed.
- City of Calgary, **Historical Property Assessments (Parcel)**, dataset 4ur7-wsgc.
- City of Calgary, **Calgary Transit Stops**, dataset muzh-c9qc.
- City of Calgary, **Parks Sites**, dataset kami-qbfh.
- City of Calgary, **Schools**, dataset fd9t-tdn2.
- Canada Mortgage and Housing Corporation, **Housing Market Information Portal**, Calgary income tables.

Source pages were checked on August 4, 2026. The archive includes the derived modeling CSV required for evaluation, not every upstream municipal extract

Installation and Execution

To run the main web interface, follow the following steps:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# ^^^ only if venv is not already created previously

python -m pip install -r requirements.txt
python -m RunROIClassifier


# Copy and paste the resulting link into the browser of your choice
```

To run the model comparison:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# ^^^ only if venv is not already created previously

python -m pip install -r requirements.txt
python -m Model.EvaluateModels –models dummy custom-rf (other models) –output-dir Artifacts/Comparison

# ^^^Models can be chosen from: dummy custom-rf custom-olr custom-ensemble sklearn

Python -m unittest discover -s tests -v
```

Note: each run's output directory should be different to avoid mixing artifacts.

Reproducibility evidence

- 224 rows with no missing model values.
- 75 Low, 74 Medium, 75 High.
- Seed, folds, hyperparameters, and features recorded.
- Dataset hash written to the manifest.
- Five automated tests pass in a fresh extraction.
- Saved and fresh-run metrics agree.
- Count and normalized matrices are included.
- All test evidence is real and out of fold.

Limitations, portability, and responsible use

Historical raw-data preparation depended on manual extracts and local utilities, so it is not represented as a one-command pipeline. Results may not transfer across cities, time periods, project types, market conditions, or assessment policies.

> **Responsible-use statement:** the output must not be used to approve, reject, price, or finance a real development. It is an educational result based on historical public-data proxies and requires expert interpretation.

References

- City of Calgary Open Data datasets c2es-76ed, 4ur7-wsgc, muzh-c9qc, kami-qbfh, and fd9t-tdn2.
- Canada Mortgage and Housing Corporation, Housing Market Information Portal.
- Group 14 project repository and generated evidence under Artifacts/Milestone2.